2025 IEEE 26th China Conference on System Simulation Technology and its Applications (CCSSTA) | 979-8-3315-4404-1/25/$31.00 ©2025 IEEE | DOI: 10.1109/IEEECONF65522.2025.11136878

Behavior Generation for Heterogeneous Agents in Urban Simulation Deduction: A Multi-Stage Approach Based on Large Language Models

Qingxin Meng
Artificial Intelligence Institute
China Electronics Technology Group Corporation
Beijing, China
meng_qx2018@163.com

Yongzhen Wang*
Scientific And Technological Innovation Center
Beijing, China
308961950@qq.com

Chun Feng
Scientific And Technological Innovation Center
Beijing, China
1442659103@qq.com

Peng Li
Artificial Intelligence Institute
China Electronics Technology Group Corporation
Beijing, China
326844536@qq.com

Xiaokai Xia
Artificial Intelligence Institute
China Electronics Technology Group Corporation
Beijing, China
xiaxiaokai@buaa.edu.cn

***Abstract*—Traditionally, constructing a professional simulation deduction script is a long-cycle process that requires collaboration between domain experts and simulation experts. The increasing demands for simulation deduction across large-scale scenarios and emergency situations highlight the need for rapid simulation scenario construction methods. Recent advances in large language models (LLMs) offer powerful new tools to address these challenges. In this work, we propose LLM-HABG, a highly portable framework that can automatically generate behavior plans for heterogeneous agents in urban simulation environments based on natural language instructions, requiring no additional post-training. It comprises four stages: task extraction, task decomposition and allocation, urban information retrieval, and behavior generation. To overcome the limitations of LLMs, such as deficient instruction-following and hallucinations, we incorporate a suite of enhancement techniques including dynamic few-shot learning, entity alignment, and constrained decoding. Experimental results demonstrate that LLM-HABG achieves robust performance even with 14B-parameter models, attaining a success rate exceeding 75%—nearly doubling baseline performance. This conversational behavior configuration paradigm empowers domain experts to interact directly with simulation systems via natural language, significantly reducing the cost and barriers associated with simulation scenario construction.**

***Keywords*—Large Language Models, Heterogeneous Agents, Behavior Generation, Urban Simulation**

I. Introduction

Simulation-based deduction plays a crucial role in verifying the feasibility of plans and supporting decision-making across domains such as urban combat and emergency response. Constructing these simulation scenarios typically demands extensive manual efforts and close collaboration between domain and simulation experts, involving translation of abstract plans into executable behaviors for heterogeneous agents—tasks that must consider temporal, spatial, and environmental interactions [1].

Despite previous attempts to streamline this process, such as behavior modeling systems with high reusability [2] and evolution-based behavior tree generation [3], existing approaches still require heavy manual design or fail to generalize flexibly across diverse scenarios.

Recent progress in large language models (LLMs) introduces new opportunities for intelligent behavior generation. LLM-based agent systems have shown success in task planning, code generation, and robotic control [4], suggesting their potential for direct natural language interaction with simulation platforms. If domain experts could configure agent behaviors through dialogue rather than coding, it would significantly lower the barrier to simulation use. As shown in Fig. 1, this shift—from expert-driven manual development to LLM-assisted configuration—represents a paradigm change in simulation scenario construction.

Research on LLM-based simulation agent behavior generation for simulation deduction purposes does not start from scratch. Studies on enabling robots to respond to natural language instructions and generate behavioral decisions in the real world share notable similarities: both emphasize high-level behavioral planning for agents, achieving complex behavioral logic by orchestrating the agents' low-level capabilities. However, when examined from the perspective of simulation deduction, this problem exhibits several distinctive characteristics:

- **Agent heterogeneity:** Comprehensive scenarios involve multiple agent types with distinct skill sets requiring systematic addressing.
- **Dual information access:** Simulation agents may use both sensor-based perception and system-level queries

*Corresponding Author

979-8-3315-4404-1/25/$31.00 ©2025 IEEE

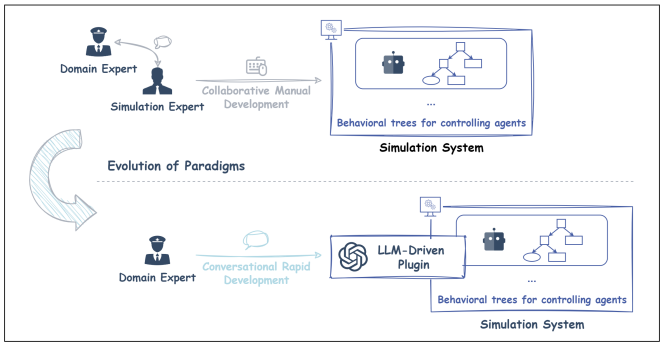


Fig. 1: Paradigm shift in simulation scenario construction: from traditional expert collaboration manual development mode to LLM-driven automated simulation scenario construction.

under rationality constraints.

- **Closed environment:** Simulations assume complete world knowledge, enabling analytical planning.
- **Instruction-oriented orchestration:** Greater focus on behavior orchestration according to predefined schemes with clearer scenario expectations.
- **Tolerant of latency:** Generation before deduction allows use of compute-heavy LLMs.

These distinctions necessitate the development of novel domain adaptation methods in the simulation field. Thus, we propose **LLM-HABG**, a multi-stage LLM-based framework for generating heterogeneous agent behaviors in urban simulations from natural language input. The system incorporates techniques such as dynamic few-shot learning, entity alignment, and constrained decoding to mitigate LLM limitations such as knowledge constraints, context length restrictions, instruction-following deficiencies, and hallucinations, and ensure reliable structured output.

Our key contributions are: (i) A unified, scalable, and practical LLM-based heterogeneous agent behavior generation framework tailored to urban simulation deduction; (ii) Integration of robust techniques to enhance behavior generation under resource-limited LLMs; (iii) An LLM-oriented urban geographic information retrieval toolkit that bridges high-level intent with spatial planning.

II. Related Work

A. Large Language Model-Driven Simulation Scene Editing

With the emergence of conversational applications based on large language models (LLMs) like ChatGPT, how to endow large language models with perception and action capabilities to enable them to complete more professional work has become one of the main research directions for further community research [5], [6]. Simulation scene editing is an important application area. Whether in autonomous driving, military simulation, or game development, both simulation experts and domain experts are required to invest substantial time in scene editing. An early exploratory work attempted to use large language models to edit autonomous driving simulation scenes [7]. In addition, recent advances in 3D urban scene generation [8], [9] have greatly improved the efficiency of real-world urban modeling. Although these initial efforts are still immature in terms of operational flexibility, generative effects and scene complexity, they have highlighted the potential of conversational approaches for simulation scene editing. Nevertheless, for the automatic generation of simulation scenarios in urban deduction contexts, the capability to generate agent behavior rules is crucial. While some LLM-driven simulation systems have partially incorporated this functionality, a practical and comprehensive framework for simulation agent behavior generation and its implementation remains lacking.

B. Robot Behavior Generation

LLM-driven robot behavior generation has emerged as an active area of research, with the primary objective of translating natural language instructions into executable robot behaviors. Some studies [10], [11] directly generate sequential behavior sequences, thereby implementing a robot workflow, achieving good results in many scenarios; there are also studies that generate control code or hierarchical behavior control structures [12]–[14], which are capable of expressing more sophisticated behavioral logic. Among these, behavior trees have become a representative structure, widely adopted due to their robust ecosystem support and advantageous properties for modular and scalable behavior modeling [15].

There are also studies [16] focusing on large language model-driven robot team behavior generation, which need to additionally consider the heterogeneity of different robot capabilities and reasonably assign tasks to individual robots in the team according to natural language instructions, thereby enabling effective task planning and collaborative behaviors for robot teams. SMART-LLM [17] introduces a framework in which LLMs decompose natural language instructions into subtasks and allocate them to heterogeneous robots according to their predefined skill sets, establishing a classic and effective paradigm. There are also studies [18] focusing on task planning capabilities for multi-robot teams in urban environments, demonstrating the powerful semantic-based planning capabilities of LLMs and their application potential in urban environments.

III. Method

With the ongoing exploration and application of large language models in related semantic parsing tasks—such as code generation and robot behavior generation—multi-stage, progressive LLM workflows have demonstrated notable advantages, including enhanced overall capability, improved output controllability, and greater ease of development and testing across a variety of tasks [17], [19], [20].

However, these multi-stage generation frameworks are typically task-specific and lack transferability across different applications. Designing a new framework requires analyzing task characteristics, decomposing problems horizontally and vertically into sub-tasks manageable for LLMs, and ensuring adequate relevant information access at each stage, thereby

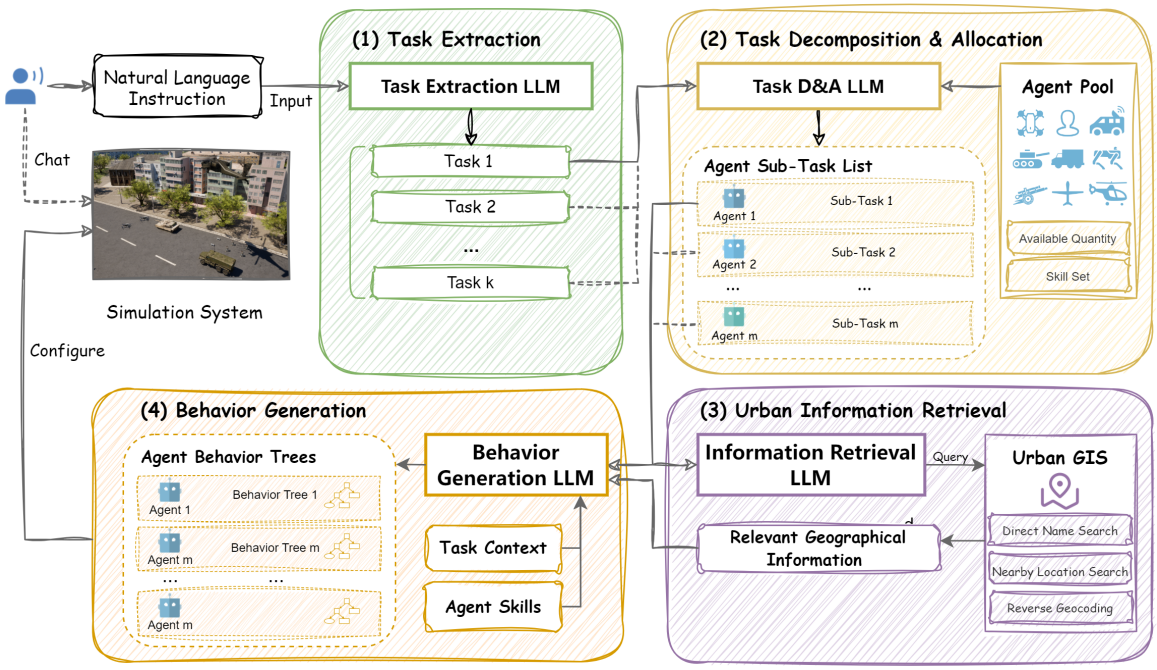


Fig. 2: **Architecture of LLM-HABG.** Users interact with the simulation system in a conversational manner to configure agent behaviors. Natural language instructions provided by users are processed through multiple stages within the framework, ultimately generating action plan for simulation agents to be configured in the simulation system.

mitigating inherent LLM limitations including knowledge gaps, context constraints, instruction-following deficiencies, and hallucinations.

This study introduces a heterogeneous agent behavior generation framework for urban simulation deduction, based on a core assumption: agents' low-level perception and action capabilities are underpinned by predefined skill repositories. LLMs accomplish dynamic orchestration and synthesis of complex behavioral logic by invoking these low-level capabilities as atomic conditions and actions for behavior tree construction.

A. Framework

LLM-HABG adopts a loosely-coupled modular architecture enabling concurrent processing across multiple tasks and agents while supporting independent optimization and testing of each component. Moreover, it supports highly flexible and dynamic context management. This design ensures linear scalability for lengthy task instructions and large-scale multi-agent simulation environments. As shown in Fig. 2, the framework contains four stages: task extraction, task decomposition and allocation, urban information retrieval, and behavior generation.

1) Task extraction: The task extraction module processes natural language instructions from humans and extracts task sequences. This is achieved by an LLM with prompt templates, aiming to: (i) filter noise in human instructions; (ii) segment lengthy inputs to make the input for the next module more standardized. The resulting task instructions are then forwarded to subsequent modules.

2) Task decomposition and allocation: Task decomposition and allocation constitutes the second stage of LLM-HABG, utilizing an LLM with carefully designed prompts and dynamic few-shot learning mechanisms. This stage assigns extracted task instructions to available simulation agents, decomposing them into specific, executable subtasks within the simulation context. The process addresses significant challenges: (i) one-to-many mapping from high-level tasks to agent subtasks, and (ii) translation of human instructions maybe in abstract or macro manner into concrete "who does what" simulation directives. Generated agent subtasks are subsequently processed by downstream modules.

3) Urban information retrieval: To support behavior generation, it is essential to identify the urban geographic information required by agent subtasks. We developed an LLM-oriented urban geographic information retrieval toolkit that provides accurate and semantically rich data, enabling LLMs to query entities such as schools, hospitals, train stations, and main roads for use in behavior generation stage. Similar designs have been shown to be closely related to final performance in LLM applications in urban scenarios [21], [22], while also mitigating issues such as hallucinations and limited numerical reasoning.

The urban geographic information retrieval toolkit provides three core functions: direct place name queries, nearby location searches, and reverse geocoding (as detailed in TABLE I). These functions are made accessible to the information retrieval LLMs via function calls.

4) Behavior generation: Behavior generation is the final stage of the heterogeneous simulation agent behavior gener-

TABLE I: **Geographic information services provided by Urban GIS.** These functions are specially designed and provided as tools for LLM calls.

Name	Implementation
Direct_Name_Search *[Name]*	Return the most similar geographic entity matched by fuzzy search.
Nearby_Location_Search *[Name, Distance, TypeTree]*	Return all geographic entities of the specified type within a given distance from the location.
Reverse_Geocoding *[Coordinates]*	Return the place name corresponding to the coordinate using reverse geocoding.

ation framework. It adopts a dynamic context management mechanism, taking dynamically retrieved urban geographic information, task context, and heterogeneous agent skill sets as input. It generates behavior trees for each agent based on their assigned subtasks, ultimately producing a complete action plan. This stage presents two primary challenges: behavior orchestration and parameter configuration.

Behavior orchestration involves assembling atomic conditions and actions from the agent skill set to construct a coherent behavior tree aligned with subtask requirements. Parameter configuration assigns appropriate values—such as height or geographic coordinates—to each condition and action, ensuring correct execution within the simulation system.

We adopt a constrained hierarchical behavior tree structure with a strictly enforced two-layer grammar: (i) Time slicing (first layer): The root selector node divides the behavior timeline into discrete intervals, each represented by a sequence node that combines a temporal condition with its corresponding action logic. (ii) Conditional branching (second layer). Within each time slot, selection nodes evaluate preconditions in priority order, enabling ”if-then-else” logic and allowing high-priority tasks (e.g., emergency responses) to preempt others for timely environmental adaptation.

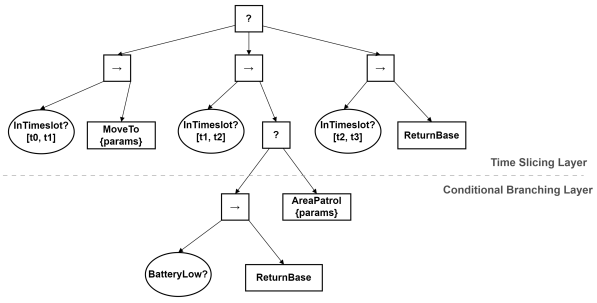


Fig. 3: **Example of the behavior tree structure.** The behavior tree is organized by time slots, with each time slot corresponding to a set of independent behavior logic.

Fig. 3 illustrates an example of the proposed behavior tree. This hierarchical structure is specifically designed for urban simulation systems, with time as the primary organizing dimension—each time interval corresponds to an independent set of behavior logic. It functions analogously to an activity schedule, specifying both what to do at a given time and how to respond to different conditions during execution.

LLMs are tasked with generating structured textual representations of this behavior tree under strict grammatical constraints. By balancing expressive power with syntactic simplicity, the structure minimizes complexity while fulfilling core task requirements, enabling stable and reliable behavior generation.

B. Dynamic Few-shot Learning

Dynamic few-shot learning is a key technique for enhancing both performance and robustness. By dynamically selecting relevant few-shot examples based on the current task instruction and agent type, this approach incorporates domain knowledge, supplies high-quality exemples, and better aligns LLM outputs with human preferences.

In LLM-HABG, each LLM module is guided by both prompt instructions and few-shot examples. Empirical results show that including contextually similar examples significantly improves the quality of the model's outputs. The dynamic few-shot learning mechanism selects the most relevant examples from a sample library based on the task at hand.

C. Entity Alignment

We employ an entity alignment mechanism to accurately map LLM-generated entity names to standardized entities present in the simulation system. In the LLM-HABG framework, entity alignment is applied to three key categories: agent names, atomic condition and action names, and geographic entity names.

This alignment mechanism prevents system failures caused by unrecognized LLM outputs. And assuming that LLM-generated names are partially correct in this case, they can most likely be corrected to the accurate names through similar matching. This is crucial for improving the overall robustness of the system on smaller parameter scale LLMs and avoiding LLM hallucinations.

D. Constrained Decoding

To ensure structural consistency of information exchanged between modules in the multi-stage generation framework, we employ constrained decoding [23].

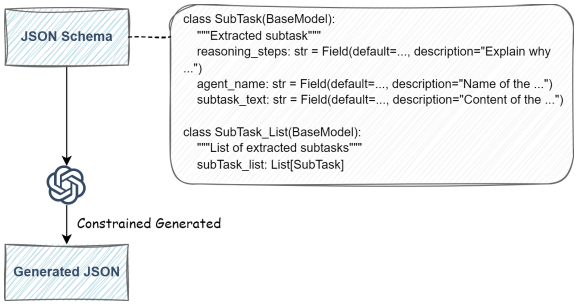


Fig. 4: Structured output generation via constrained decoding. The process follows $P(X|C) = \prod_{i=1}^{n} p(x_i|x_{<i}, C)$, where x_i is the current token, $x_{<i}$ represents preceding tokens, and C denotes the constraint set.

Constrained decoding refers to checking the vocabulary and filtering out words that violate specified structures during the

decoding stage of language models, thereby guaranteeing the structure of LLM outputs. The advantage of this method is that it enables LLM structured output without hard prompts, greatly reducing dependence on the models's instruction-following capabilities [24].

Fig. 4 illustrates the structured output process, using the generation of the Agent Sub-Task List in the second stage as an example.

IV. Experiment

To verify the effectiveness of LLM-HABG, we conducted experiments in a simulation system specifically designed for urban environment deduction. This system has rich agent modeling assets and is based on real city scene modeling.

A. Typical Case Analysis

A typical case is used to demonstrate the effectiveness and practicality of LLM-HABG. The input instruction is as follows:

"At 02 minutes, the red quadrotor reconnaissance aircraft in standby status takes off to reconnaissance in the 300m area near Y Elementary School, focusing on covering the school airspace and surrounding main roads, returning to the takeoff position after 30 minutes. At 05 minutes, unmanned vehicle No. 1 departs from the assembly area at 30km/h speed to approach near Y Police Station, and if attacked, seek obstacles to hide."

The multi-stage processing flow of LLM-HABG is outlined below:

1) Task extraction: Two tasks related to the quadrotor reconnaissance aircraft and unmanned vehicle are extracted and forwarded for sequential processing:
"Task 1: At 02 minutes, the red quadrotor reconnaissance aircraft ...
Task 2: At 05 minutes, red unmanned vehicle No. 1..."

2) Task decomposition and allocation: For Task 1, the system identifies all deployed red quadrotor reconnaissance aircraft in standby status (numbered 0, 1, 3, 4) and assigns subtasks to each:
"QuadRecon__0, at 02 minutes go to reconnaissance in the 300m area near Y Elementary School, focusing on covering the school airspace; return to takeoff position after 30 minutes;
QuadRecon__1, at 02 minutes go to reconnaissance in the 300m area near Y Elementary School, focusing on covering surrounding main roads; return to takeoff position after 30 minutes;
QuadRecon__3, at 02 minutes go to reconnaissance in other areas within 300m near Y Elementary School; return to takeoff position after 30 minutes;
QuadRecon__4, ... (same as QuadRecon__3)"

For Task 2, one subtask is generated:
"Unmanned_Vehicle__1, at 05 minutes depart from assembly area..."

3) Urban information retrieval: For Task 1, the LLM processes each agent subtask, identifying geographic entities such as "Y Elementary School", "Y Elementary School airspace", and "main roads within 300m around Y Elementary School", and generates corresponding queries to Urban GIS:
"Direct_Name_Search['Y Elementary School']
Direct_Name_Search['Y Elementary School airspace']
Nearby_Location_Search['Y Elementary School', 300, 'road', 'primary']"

For Task 2, the query is:
"Direct_Name_Search['Y Police Station']"

The resulting geographic entities are collected for use in the behavior generation stage.

4) Behavior generation: For Task 1, the LLM processes each agent subtask:
"QuadRecon__0, queries the quadrotor reconnaissance aircraft skill set. The LLM leverages relevant few-shot prompts, identifies conditions InTimeslot, BatteryLow, and actions AreaPatrol, ReturnBase as pertinent, and uses them to construct the behavior tree, configuring patrol area parameters based on the retrieved geographic information. The generated behavior tree is shown in Fig. 5a.
QuadRecon__1, ... ; QuadRecon__3, ... ; QuadRecon__4, ..."

For Task 2, the LLM processes the subtask:
"Unmanned_Vehicle__1, queries the unmanned vehicle skill set. The LLM leverages relevant few-shot prompts, identifies conditions InTimeslot, AttackDetect, and actions MoveTo, Hide as relevant, and generates the behavior tree and parameters by combining task context and geographic information. The generated behavior tree is shown in Fig. 5b."

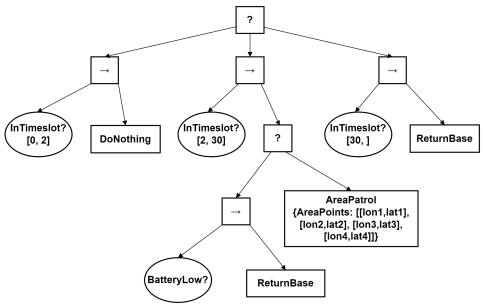
The final action plan is deployed in the simulation system, with action areas visualized in Fig. 5c. QuadRecon__0's area covers Y Elementary School, QuadRecon__1 covers the main road Z Road, and QuadRecon__3 and QuadRecon__4 are assigned to surrounding areas.

In this case, the LLM-HABG framework effectively interprets complex natural language instructions, decomposes and assigns subtasks to heterogeneous agents, dynamically queries agent skill sets, sample repository and geographic information, and generates collaborative action plans in the form of behavior trees based on task context.

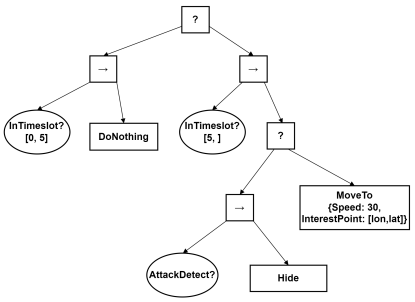
This LLM-driven approach demonstrates distinct advantages. Leveraging broad prior knowledge and contextual reasoning, LLMs can generalize to diverse agents and task instructions through few-shot or even zero-shot learning.

Moreover, LLMs can effectively identify and utilize semantic information in natural language for planning. For example, "main roads" and "school airspace" in task instructions are utilized by LLMs as task objectives to plan collaborative actions for quadrotor reconnaissance aircraft teams. Furthermore, LLMs can utilize rich semantic labels in geographic data to identify relevant locations and generate plausible patrol areas and routes. This semantic-driven planning approach offers greater flexibility and adaptability than traditional methods.

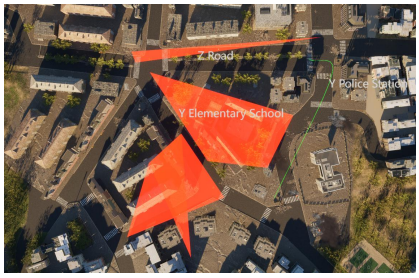
However, several limitations remain. LLMs lack precise spatial computation capabilities, often producing suboptimal



(a) Behavior tree of QuadRecon__0



(b) Behavior tree of Unmanned_Vehicle__1



(c) Quadrotor reconnaissance aircraft action area and unmanned vehicle action route drawn in the simulation system.

Fig. 5: Behavior configuration results for the case.

patrol areas—e.g., with overlaps, irregular shapes, or unmet range constraints. They also struggle to independently generate complete obstacle-avoiding routes, which still require low-level path planning algorithms. Additionally, producing high-quality behavior trees often relies on domain expertise, which is challenging for LLMs to replicate in zero-shot scenarios.

B. Performance Evaluation and Comparative Analysis

We evaluate LLM-HABG against a Reasoning & Summarizing (R&S) baseline using a test set of 40 natural language instructions authored by domain experts. The test set is categorized into simple and complex types, with 20 samples in each category.

Reasoning & Summarizing is a general-purpose method that involves reasoning followed by summarization. In the first step, relevant elements such as agent skill sets, output format requirements, output examples, and geographic information are cropped based on maximum length constraints and combined with the natural language instructions as input to LLMs, prompting them to directly reason and generate action plans for heterogeneous agents. The second step summarizes the structured behavior from the output of the reasoning step. We tested both methods using GPT-4o mini and Qwen2.5-14B models, representing advanced commercial and mid-scale open-source LLMs respectively.

To evaluate the LLM-generated heterogeneous agent action plans, we adopted two complementary approaches. First, the execution success rate measures whether the generated plans can be correctly parsed and executed by the system. Second, expert evaluation was conducted by inviting five simulation experts and five domain experts to evaluate various aspects of the generated outputs, focusing primarily on the following criteria:

- **Task Allocation Accuracy**: Whether tasks are correctly assigned to the appropriate agents.
- **Behavior Orchestration Accuracy**: Whether the structured behavior tree and selected nodes fulfill the instruction requirements.
- **Behavior Parameters Configuration Accuracy**: Whether the parameter settings of behavior tree nodes align with the instruction requirements.
- **Overall Success Rate**: Whether the generated action plan successfully satisfies the instruction.

TABLE II presents the consensus evaluations of participating experts on the generated results for the instruction dataset. The results indicate that LLM-HABG significantly outperforms across all metrics, particularly with the Qwen2.5-14B model and complex instruction samples, achieving nearly a twofold improvement. This performance gain stems from the combined effect of the multi-stage framework, dynamic few-shot learning, entity alignment, and constrained decoding. Notably, Qwen2.5-14B achieved 100% execution success and overall success rates of 16/20 and 14/20 on simple and complex instructions, respectively.

In contrast, the R&S method showed poor performance in parameter configuration accuracy (Param) and overall success rate (SR), especially under the Qwen2.5-14B model, where only 17 out of 40 samples were successful. Analysis of failure cases reveals three main causes: (i) The R&S method lacks dynamic query capabilities, leading to loss of critical information during input cropping; (ii) The absence of a divide-and-conquer framework significantly lowers success rates when processing complex instructions; and (iii) The lack of alignment mechanisms leads to parsing errors. Particularly on small parameter scale LLMs, due to their limited instruction-following ability, difficulty in processing lengthy contexts may ultimately generate incorrect agent names, incorrect behavior parameter types, or unusable agent skills.

V. SUMMARY

This research addresses heterogeneous agent simulation behavior generation in urban environments through LLM-HABG, a multi-stage framework leveraging large language models. Its modular and scalable design provides a solid foundation for future functional extensions and performance optimizations, capable of adapting to ever-growing simulation scale requirements.

Experimental validation demonstrates significant improvements in success rates, particularly for smaller open-source models, effectively enabling natural language-driven simulation scenario construction. Despite these advances, several limitations remain, including dependency on carefully annotated geographic data, reliance on high-quality low-level behavior

TABLE II: **Experimental evaluation results.** Shows the performance of LLM-HABG and Reasoning & Summarizing (R&S) methods using different base models. Each evaluation metric is divided into Easy and Difficult instruction difficulty results, including execution success rate (Exec), task allocation accuracy (Alloc), behavior orchestration accuracy (Orch), behavior parameter configuration accuracy (Param), and overall success rate (SR).

Method		Accuracy (%)									
		Exec		Alloc		Orch		Param		SR	
		Easy	Diff	Easy	Diff	Easy	Diff	Easy	Diff	Easy	Diff
LLM-HABG	Qwen2.5-14B	**20/20**	**20/20**	20/20	19/20	19/20	17/20	15/20	14/20	**16/20**	**14/20**
	GPT-4o mini	20/20	20/20	20/20	19/20	19/20	17/20	19/20	18/20	18/20	16/20
R & S	Qwen2.5-14B	15/20	12/20	20/20	17/20	15/20	12/20	12/20	10/20	10/20	7/20
	GPT-4o mini	19/20	17/20	20/20	19/20	17/20	15/20	15/20	13/20	14/20	10/20

models, and challenges with complex spatial reasoning due to current LLM capabilities.

Looking forward, there are two promising research directions: (i) integrating vision-language models to enhance spatial understanding and enable agent-level perception through visual input; (ii) developing standardized behavior tree generation datasets with automated evaluation frameworks to support method benchmarking and LLM fine-tuning.

In summary, this study outlines a viable technical pathway for LLM-based agent behavior generation in simulations, contributing to the advancement of intelligent simulation systems. In scenarios such as local conflicts or emergencies, the need for rapid simulation-based decision support is critical. Natural language interaction with simulation platforms offers substantial potential for real-world applications.

REFERENCES

[1] Z. Dong, Z. Hu, Z. Liu, and H. Zhou, "A Review of Intelligent Generation of Combat Simulation Scenarios," *Journal of System Simulation*, pp. 1–19, 2025.

[2] G. Zhang and y. Ji, "Behavior Modeling of Marine Warfare Simulation with Highly Reused Behavioral Resources," *Ship Electronic Engineering*, vol. 44, no. 8, pp. 106–112, 2024.

[3] M. Colledanchise, R. Parasuraman, and P. Ögren, "Learning of Behavior Trees for Autonomous Agents," *IEEE Transactions on Games*, vol. 11, no. 2, pp. 183–189, Jun. 2019.

[4] R. Sapkota, K. I. Roumeliotis, and M. Karkee, "AI agents vs. Agentic AI: A conceptual taxonomy, applications and challenges," https://arxiv.org/abs/2505.10468v4, May 2025.

[5] Z. Xi, W. Chen, X. Guo, W. He, Y. Ding, B. Hong, M. Zhang, J. Wang, S. Jin, E. Zhou, R. Zheng, X. Fan, X. Wang, L. Xiong, Y. Zhou, W. Wang, C. Jiang, Y. Zou, X. Liu, Z. Yin, S. Dou, R. Weng, W. Cheng, Q. Zhang, W. Qin, Y. Zheng, X. Qiu, X. Huang, and T. Gui, "The Rise and Potential of Large Language Model Based Agents: A Survey," Sep. 2023.

[6] T. R. Sumers, S. Yao, K. Narasimhan, and T. L. Griffiths, "Cognitive Architectures for Language Agents," Mar. 2024.

[7] Y. Wei, Z. Wang, Y. Lu, C. Xu, C. Liu, H. Zhao, S. Chen, and Y. Wang, "Editable Scene Simulation for Autonomous Driving via Collaborative LLM-Agents," Jun. 2024.

[8] Y. Shang, Y. Lin, Y. Zheng, H. Fan, J. Ding, J. Feng, J. Chen, L. Tian, and Y. Li, "UrbanWorld: An Urban World Model for 3D City Generation," Oct. 2024.

[9] M. Zhou, Y. Wang, J. Hou, S. Zhang, Y. Li, C. Luo, J. Peng, and Z. Zhang, "SceneX: Procedural Controllable Large-scale Scene Generation," Dec. 2024.

[10] C. H. Song, J. Wu, C. Washington, B. M. Sadler, W.-L. Chao, and Y. Su, "LLM-Planner: Few-Shot Grounded Planning for Embodied Agents with Large Language Models," Mar. 2023.

[11] Y. Chen, W. Cui, Y. Chen, M. Tan, X. Zhang, D. Zhao, and H. Wang, "RoboGPT: An intelligent agent of making embodied long-term decisions for daily instruction tasks," Sep. 2024.

[12] I. Singh, V. Blukis, A. Mousavian, A. Goyal, D. Xu, J. Tremblay, D. Fox, J. Thomason, and A. Garg, "ProgPrompt: Generating Situated Robot Task Plans using Large Language Models," Sep. 2022.

[13] Y. Cao and C. S. G. Lee, "Robot Behavior-Tree-Based Task Generation with Large Language Models," Feb. 2023.

[14] J. Ao, F. Wu, Y. Wu, A. Swikir, and S. Haddadin, "LLM as BT-Planner: Leveraging LLMs for Behavior Tree Generation in Robot Task Planning," Sep. 2024.

[15] M. Colledanchise and P. Ögren, *Behavior Trees in Robotics and AI: An Introduction*, Jul. 2018.

[16] P. Li, Z. An, S. Abrar, and L. Zhou, "Large language models for multi-robot systems: A survey," Feb. 2025.

[17] S. S. Kannan, V. L. N. Venkatesh, and B.-C. Min, "SMART-LLM: Smart multi-agent robot task planning using large language models," Mar. 2024.

[18] P. Gupta, D. Isele, E. Sachdeva, P.-H. Huang, B. Dariush, K. Lee, and S. Bae, "Generalized mission planning for heterogeneous multi-robot teams via LLM-constructed hierarchical trees," Jan. 2025.

[19] X. Xie, G. Xu, L. Zhao, and R. Guo, "OpenSearch-SQL: Enhancing text-to-SQL with dynamic few-shot and consistency alignment," Feb. 2025.

[20] M. Pourreza and D. Rafiei, "DIN-SQL: Decomposed In-context learning of text-to-SQL with self-correction," Nov. 2023.

[21] Y. Tang, Z. Wang, A. Qu, Y. Yan, Z. Wu, D. Zhuang, J. Kai, K. Hou, X. Guo, H. Zheng, T. Luo, J. Zhao, Z. Zhao, and W. Ma, "ITINERA: Integrating spatial optimization with large language models for open-domain urban itinerary planning," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*, 2024, pp. 1413–1432.

[22] W. Chen, M.-W. Su, N. Mehjabin, and M. L. Cummings, "Can LLMs plan paths in the real world?" Dec. 2024.

[23] Y. Dong, C. F. Ruan, Y. Cai, R. Lai, Z. Xu, Y. Zhao, and T. Chen, "XGrammar: Flexible and efficient structured generation engine for large language models," May 2025.

[24] S. Tharsis, "Taming LLMs: A Practical Guide to LLM Pitfalls with Open Source Software," 2025.